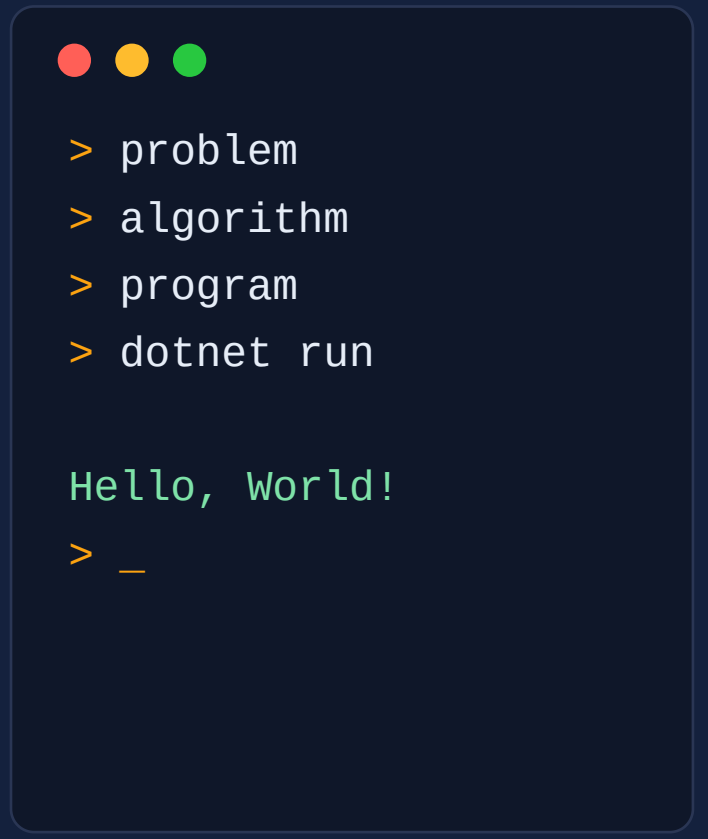
>_ MCMP113

Introduction to Basic Algorithms and Programming

Week 1 — Course Introduction · What Is an Algorithm? · Problem-Solving Steps



Asst. Prof. Hasan Oğuz, PhD
Istanbul Okan University · Vocational School
2026–2027 Fall Semester

Your Instructor

INSTRUCTOR

Asst. Prof. Hasan Oğuz, PhD

CONTACT

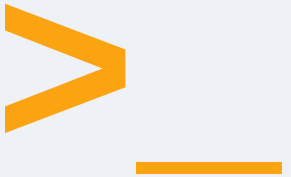
Details are given in class and on the course page

OFFICE HOURS

Announced in class · or by appointment

COURSE GROUP

Announced in class



Join the course group today

Announcements · slides · sample codes · homework

What You Will Be Able to Do

By the end of the semester you will be able to …

Think algorithmically

Break a real problem into precise, finite, ordered steps.

Draw & read flowcharts

Express an algorithm visually before writing any code.

Write C# console apps

Use variables, operators, decisions and loops correctly.

Work with arrays

Store, search, sort and process collections of data.

Structure with functions

Split programs into reusable methods with parameters.

Test and debug

Trace code by hand and find errors systematically.

15-Week Plan

1	**Course introduction · Algorithm concept · Problem-solving steps**
2	Flowcharts and their symbols · Basic flowchart examples
3	Programming languages · Variables · Data types · Constants
4	Arithmetic, logical and relational operators
5	Decision structures: if-else, if-else if, switch-case
6	Loops: the for loop
7	Loops: while and do-while
8	**MIDTERM EXAM**
9	One-dimensional arrays: declaration, assignment, element access
10	Array operations: searching, sorting, finding max/min
11	Multi-dimensional arrays (matrices)
12	Functions: definition, calling, parameter passing
13	Value-returning vs. void functions · Scope
14	Basic file operations · Semester review
15	**FINAL EXAM**

Today is highlighted. Weeks 3 onward: every lecture comes with sample console-app code.

How You Are Graded · What You Need

40%

Midterm · Week 8

60%

Final · Week 15

- Attendance follows university regulations.
- Exams test reasoning: tracing, flowcharts and writing code by hand.
- Weekly exercises are ungraded — but they are the exam preparation.

Software (free) — install before Week 3



.NET 10 SDK (LTS)

`dotnet.microsoft.com/download`



Visual Studio Code

`code.visualstudio.com`



C# Dev Kit extension

`VS Code → Extensions → “C# Dev Kit”`

```
> dotnet --version
10.0.xxx
```



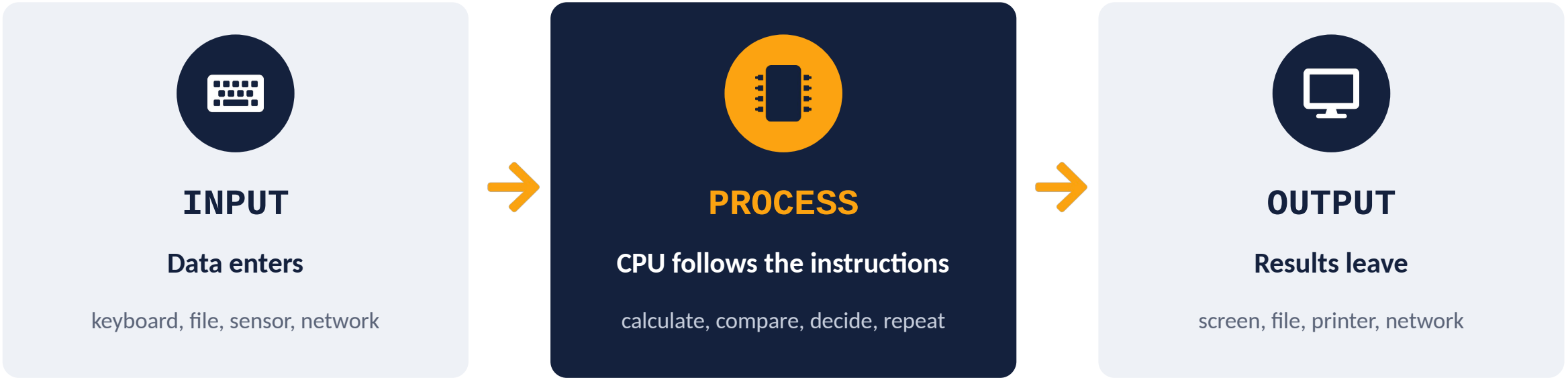
PART 1

How a Computer Solves Problems

Input → Process → Output, and why the computer needs exact instructions

>_ THE BIG PICTURE

Every Program: Input → Process → Output



Memory (RAM) holds both the data and the instructions while the program runs (von Neumann, 1945). Turn the power off and RAM is empty; files on disk remain.

>_ THE KEY INSIGHT

A Computer Is Extremely Fast — and Completely Literal

~10^9 simple operations per second on a single core

0 guesses about what you “meant”

4 kinds of instruction: arithmetic, comparison, data movement, jump



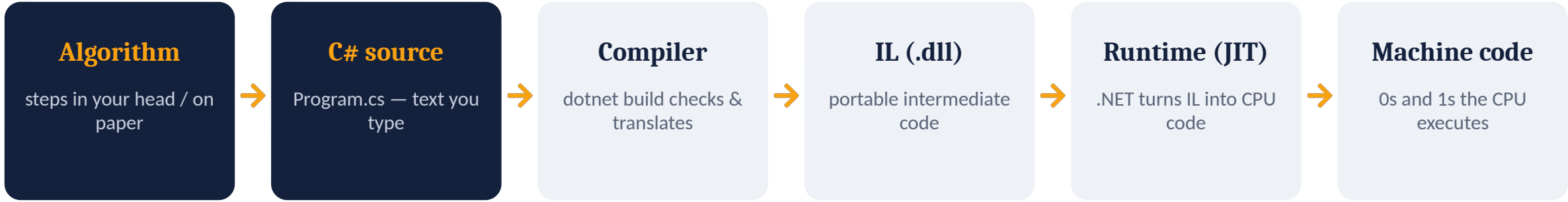
Therefore …

The hard part of programming is not typing code.

It is deciding the exact, unambiguous sequence of steps that solves the problem — the algorithm.

The language (C#) only writes those steps down in a form the machine can execute.

From Idea to Running Program



YOU write the first two boxes · **TOOLS** do the rest automatically when you type dotnet run

Same task — "put 5 in a register, add 3" — at three levels

Machine code (x86, hex)

```
B8 05 00 00 00
83 C0 03
```

What the CPU really reads. Unreadable for humans.

Assembly

```
mov eax, 5
add eax, 3
```

One mnemonic per machine instruction. CPU-specific.

High-level: C#

```
int sum = 5 + 3;
```

Close to human thinking. Runs on any OS with .NET.



PART 2

What Is an Algorithm?

A precise, finite recipe that turns input into the desired output

>_ ACTIVITY · 5 MIN

The Exact Instructions Challenge

Live demo: make a sandwich

- A volunteer plays the “computer” and does exactly what is said, nothing more.
- The class dictates the steps for making a sandwich, one instruction at a time.
- “Put the peanut butter on the bread” → the closed jar lands on the loaf.
- Fix the instructions, run again, repeat until it works.

While you observe, note down:

1. Which instruction was ambiguous?
2. Which step was missing?
3. Which step was in the wrong order?
4. Did the “algorithm” ever end?
5. How did the class fix it?

>_ DEFINITION

Algorithm

An algorithm is a **finite** sequence of **well-defined** steps that transforms a given **input** into the desired **output** and then stops.

Where does the word come from?

Muhammad ibn Mūsā al-Khwārizmī (c. 780–850), a mathematician of the House of Wisdom in Baghdad, wrote the book that taught Europe the Hindu–Arabic decimal system. His name, Latinised as “Algoritmi”, became algorithm; the title of his other book (al-jabr) gave us algebra.

Algorithm vs. Program

`Algorithm`

the idea — language-independent; written in words, pseudocode or a flowchart

`Program`

one implementation of the idea in a specific language (C#, Python …)

`1 → many`

one algorithm can be coded in any language

>_ MOTIVATION

Algorithms Are Already Running Your Day

Navigation

Shortest-path algorithms (Dijkstra, 1959) pick your route on Google Maps / Yandex.

Web search

Ranking algorithms order billions of pages in a fraction of a second.

Banking & ATM

PIN check, balance rules, fraud detection — all precise decision steps.

Compression

MP3, JPEG, ZIP: algorithms that shrink data you send and store.

Communication

Every Wi-Fi / 4G packet carries a checksum so errors are detected.

Control

Traffic lights, elevators, washing machines follow fixed algorithms.

Question for you: name one algorithm you personally executed this morning.

>_ FIVE PROPERTIES (KNUTH)

What Makes a Set of Steps an Algorithm?

Finiteness	It must stop after a finite number of steps.	*"Keep stirring" → stir until when?*
Definiteness	Every step is precise and unambiguous.	*"Add some sugar" → how much?*
Input	Zero or more clearly specified inputs.	*Which bread? Which knife?*
Output	At least one result that relates to the input.	*A sandwich, not an empty plate.*
Effectiveness	Every step is basic enough to actually be carried out.	*"Make it tasty" is not a step.*

>_ QUICK QUIZ · 3 MIN

Algorithm or Not?



1. Shampoo bottle: “Lather. Rinse. Repeat.”



2. The long-division method you learned in primary school.



3. Print the largest prime number.



4. Recipe: “Add salt to taste.”



5. Find a word in a paper dictionary: open near its first letter, then move forward or back.



6. Choose the best football team in the world.

For each: yes or no? If no — which of the five properties fails?

Algorithm or Not? — Answers

1. Lather. Rinse. Repeat.

Never stops — fails finiteness.

2. Long division

Finite, precise, effective. A classic algorithm.

3. Largest prime

There is no largest prime (Euclid): it never finishes, no output.

4. Salt to taste

“To taste” is not precise — fails definiteness.

5. Dictionary look-up

Ends in finitely many steps: the book is finite and ordered.

6. Best football team

No agreed criterion — not definite, not effective.

Bonus: item 5 is the idea behind binary search — you will program it in Week 10.

Brewing Turkish Tea — Precisely

1. Fill the kettle (bottom pot) with 1 L of water.
2. Put 4 teaspoons of black tea in the teapot (top pot).
3. Place the teapot on the kettle and turn the stove on.
4. **WHILE the water is not boiling: wait 1 minute.**
5. Pour boiling water into the teapot until it is ¾ full.
6. Lower the heat and wait 15 minutes.
7. **IF the guest wants it strong: fill the glass ⅔ with tea,**
   ELSE: fill the glass ⅓ with tea.
8. Top up the glass with hot water. STOP.



Colour key

Sequence

do steps in order

Selection

choose a path (IF)

Repetition

repeat (WHILE)

Three Structures Are Enough

Sequence

Steps executed one after another, top to bottom.

```
read a
read b
sum ← a + b
print sum
```

C# in Weeks 3–4

Selection

Choose between paths depending on a condition.

```
if age ≥ 18
   print "adult"
else
   print "minor"
```

C# in Week 5

Repetition

Repeat a block while a condition holds.

```
i ← 1
while i ≤ 10
   print i
   i ← i + 1
```

C# in Weeks 6–7

Four Ways to Write the Same Algorithm

Task: read a number and print its absolute value.

Natural language

Read the number. If it is negative, remove its minus sign. Show the number.

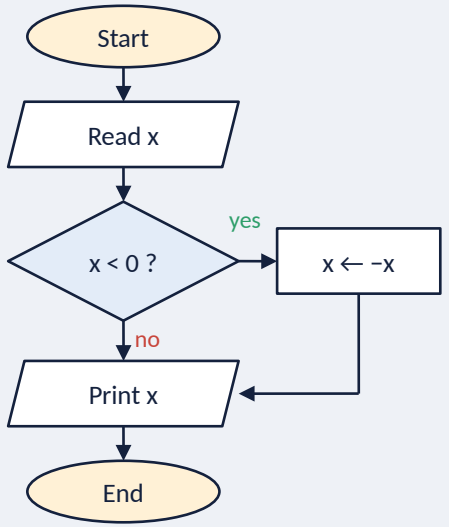
Easy for anyone — but often ambiguous.

Pseudocode

```
START
READ x
IF x < 0 THEN
   x ← −x
END IF
PRINT x
END
```

Precise and tool-free. Our main tool on paper.

Flowchart



Visual; shows branches clearly. Week 2.

C# code

```
string s =
  Console.ReadLine();
int x = int.Parse(s);
if (x < 0)
    x = -x;
Console.WriteLine(x);
```

Executable — but strict syntax. Week 3+.

>_ REFERENCE CARD · KEEP THIS

Pseudocode Conventions for MCMP113

Keyword	Meaning	Example
START … END	beginning and end of the algorithm	`START … END`
READ x	input: get a value from the user	`READ age`
PRINT …	output: show a value or a message	`PRINT "Total:", sum`
x ← expression	assignment: evaluate right side, store in x	`total ← total + price`
IF … ELSE … END IF	selection (THEN after the condition)	`IF age ≥ 18 THEN …`
WHILE cond … END WHILE	repeat while the condition is true	`WHILE i ≤ N`
FOR i ← a TO b … END FOR	repeat a counted number of times	`FOR i ← 1 TO 10`
+ − × / mod	arithmetic; mod = remainder	`17 mod 5 = 2`
= ≠ < ≤ > ≥	comparison (true / false)	`IF x ≠ 0 THEN …`
AND OR NOT	combine conditions	`IF a > 0 AND b > 0`



The #1 beginner trap

In pseudocode:

```
←  assigns
=  compares
```

In C#:

```
=   assigns
==  compares
```

x ← x + 1 is not an equation: it means “new x = old x + 1”.



PART 3

Problem-Solving Steps

From a vague request to a tested solution

>_ THE METHOD

Five Steps of Problem Solving



1. Understand

What is given? What is asked? Constraints?

2. Design

Write the algorithm: steps, pseudocode, flowchart.

3. Implement

Translate the algorithm into C#.

4. Test & Debug

Run with normal, edge and wrong inputs.

5. Document

Comments, clear names, notes for others.

It is a cycle, not a line. A failed test sends you back to Design — or even to Understand. Beginners spend most of their time in Step 3; professionals spend most of it in Steps 1–2, which is why their Step 4 is short.

Average of Three Exam Grades

Problem: A student enters three exam grades (0–100). Show the average and whether the student passed (average ≥ 50).

INPUT	PROCESS	OUTPUT
g1, g2, g3 (numbers, 0–100)	avg ← (g1 + g2 + g3) / 3 compare avg with 50	avg “PASSED” or “FAILED”

Questions to ask before designing (the “edge cases”):

- Can a grade be 72.5, or only whole numbers?
- What if the user types 120 or −5?
- Is exactly 50 a pass?
- Should all exams have equal weight?

From Plain Steps to Pseudocode

Plain-language steps

1. Start.
2. Read the first, second and third grades.
3. Add the three grades and divide by 3.
4. Show the average.
5. If the average is at least 50, show “PASSED”; otherwise show “FAILED”.
6. Stop.

Pseudocode

```
START
  READ g1, g2, g3
  avg ← (g1 + g2 + g3) / 3
  PRINT avg
  IF avg ≥ 50 THEN
     PRINT "PASSED"
  ELSE
     PRINT "FAILED"
  END IF
END
```

Pseudocode = structured English. No semicolons, no compiler — but precise enough to translate line-by-line into C#.

Choosing Good Test Cases

Kind of test	g1, g2, g3	Expected result
Normal	`60, 70, 80`	70.00 · PASSED
Boundary	`50, 50, 50`	50.00 · PASSED
Just below	`49, 50, 50`	49.67 · FAILED
Maximum	`100, 100, 100`	100.00 · PASSED
Minimum	`0, 0, 0`	0.00 · FAILED
Out of range	`120, 70, 80`	**should be rejected — our algorithm prints 90.00!**
Wrong type	`abc, 70, 80`	**C# program crashes (FormatException)**



"Program testing can be used to show the presence of bugs, but never to show their absence."

— Edsger W. Dijkstra

So choose tests that are most likely to break the algorithm.

Sum of 1 … N — with a Trace Table

```
START
  READ N
  sum ← 0
  i ← 1
  WHILE i ≤ N
     sum ← sum + i
     i ← i + 1
  END WHILE
  PRINT sum
END
```

Trace for N = 5

step	i	i ≤ N ?	sum
start	1	–	0
1	1	yes	1
2	2	yes	3
3	3	yes	6
4	4	yes	10
5	5	yes	15
exit	**6**	**no**	**15**



Same problem, different algorithm: Gauss's formula sum = N × (N + 1) / 2 gives 15 in one step. The loop needs N steps; for N = 1 000 000 000 that is a real difference. Choosing the better algorithm is a design decision, not a coding one.

Guess My Number (1 … 1000): Two Algorithms

A · Try every number

`1, 2, 3, 4, …`

1000

guesses in the worst case

B · Always guess the middle

`“higher” / “lower” halves the range`

10

guesses at most, because $2^{10} = 1024 \geq 1000$

Strategy B, secret number = 737

500	750	625	687	718	734	742	738	736	737
↑	↓	↑	↑	↑	↑	↓	↓	↑	✓

range size	100	1 000	1 000 000	1 000 000 000
A (worst)	100	1 000	1 000 000	1 000 000 000
B (worst)	7	10	20	30

>_ A 2300-YEAR-OLD ALGORITHM

Euclid's Greatest Common Divisor

Written down around 300 BC in Euclid's Elements — still used today inside every secure (HTTPS/RSA) connection.

```
START
  READ a, b
  WHILE b ≠ 0
     r ← a mod b
     a ← b
     b ← r
  END WHILE
  PRINT a
END
```

`mod = remainder of division  (17 mod 5 = 2)`

Trace: GCD(48, 18)

a	b	r = a mod b
48	18	12
18	12	6
12	6	0
6	**0**	**stop → GCD = 6**

Check all five properties: Is it finite? (b strictly decreases.) Definite? Input? Output? Effective?

T.C. Kimlik No: the Check-Digit Algorithm

The rules

1. 11 digits d1 d2 … d11, and d1 ≠ 0
2. d10 = (7 × (d1+d3+d5+d7+d9) − (d2+d4+d6+d8)) mod 10
3. d11 = (d1 + d2 + … + d10) mod 10

Why does it exist?

- Any single mistyped digit is detected instantly — without asking a database.
- Same idea: IBAN (mod 97), EAN-13 barcodes, credit cards (Luhn).

Trace with the made-up first nine digits 1 2 3 4 5 6 7 8 9

1	2	3	4	5	6	7	8	9	5	0

```
odd  positions: 1+3+5+7+9 = 25
even positions: 2+4+6+8   = 20
d10 = (7×25 − 20) mod 10 = 155 mod 10 = 5
d11 = (45 + 5) mod 10 = 0
```

Bad Algorithm → Good Algorithm

Problem	Bad	Good
Ambiguous step	`Divide the numbers.`	`avg ← (g1 + g2 + g3) / 3`
No termination	`Keep asking for a password.`	`Ask at most 3 times, then lock.`
Missing edge case	`result ← a / b`	`IF b = 0 THEN print "error" ELSE result ← a / b`
Wrong order	`PRINT sum, then sum ← a + b`	`sum ← a + b, then PRINT sum`
Undefined input	`Read the grade.`	`Read a grade between 0 and 100; reject others.`

The Same Algorithm, Written in C#

```
Console.Write("Grade 1: ");
double g1 = double.Parse(Console.ReadLine());
Console.Write("Grade 2: ");
double g2 = double.Parse(Console.ReadLine());
Console.Write("Grade 3: ");
double g3 = double.Parse(Console.ReadLine());

double avg = (g1 + g2 + g3) / 3;
Console.WriteLine($"Average: {avg:F2}");

if (avg >= 50)
    Console.WriteLine("PASSED");
else
    Console.WriteLine("FAILED");
```

Notice

- Every pseudocode line maps to one or two C# lines.
- READ → Console.ReadLine()
- PRINT → Console.WriteLine()
- IF … ELSE → if (…) … else …
- You do not need to understand the syntax yet — only that the algorithm came first.

>_ IN-CLASS · GROUPS OF 2–3 · 25 MIN

Your Turn: Write the Algorithm

A Even or odd

Read an integer; print whether it is even or odd.

`selection`

B Leap year

Read a year; leap if divisible by 4 and not by 100 — or divisible by 400.

`selection`

C ATM withdrawal

Balance and amount given; allow only if enough money and a multiple of 10.

`selection`

D Countdown

Read N; print N, N−1, …, 1, then “Go!”.

`repetition`

E Guess the number

The user guesses a secret number; say “higher/lower” until correct.

`repetition`

F Temperature

Convert Celsius to Fahrenheit: F = C × 9 / 5 + 32.

`sequence`

For each: IPO table → numbered steps → pseudocode → trace with two test inputs (one normal, one edge case).

>_ HOMEWORK · DUE BEFORE WEEK 2

Practice + Setup

1 · Algorithms (on paper)

- Finish the two in-class exercises your group did not do.
- Write an algorithm for a daily task of your choice with at least one IF and one WHILE.
- Trace Euclid's GCD for (84, 36) in a table.
- Post a photo of your pages to the course group.

2 · Install the tools

- Install .NET 10 SDK and VS Code + C# Dev Kit.
- Open a terminal and run the three commands below.
- Screenshot the output → the course group.

```
> dotnet --version
> dotnet new console -n Hello
> cd Hello
> dotnet run
```

>_ SUMMARY

Key Takeaways

01 An algorithm is a finite, precise recipe from input to output.

02 Five properties: finiteness, definiteness, input, output, effectiveness.

03 Everything is built from sequence, selection and repetition.

04 Solve in five steps: understand → design → implement → test → document.

05 Design first, code second. Trace tables catch errors before the computer does.

NEXT WEEK

Flowcharts & their symbols



MCMP113
Course materials